

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and
Commerce

Department of Information Technology (Bsc.IT Semester III)
Data Structures Practical

Practical - V

Roll No. :S051	Name:RAHUL YADAV
Class :SYIT	Batch : 1
Date of Assignment :22/07/2026	Date/Time of Submission :22/07/2026

Binary Search Tree: Write a program to:

- a. Create a binary search tree.

CODE:

```

class TreeNode:
    def __init__(self,data):
        self.data = data
        self.left = None
        self.right = None

root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')
nodeC = TreeNode('C')
nodeD = TreeNode('D')
nodeE = TreeNode('E')
nodeF = TreeNode('F')
nodeG = TreeNode('G')
nodeH = TreeNode('H')

root.left = nodeA
root.right = nodeB

nodeA.left = nodeC
nodeA.right = nodeD

nodeB.left = nodeE
nodeB.right = nodeF

nodeF.left = nodeG
nodeF.right = nodeH
#Test
print("root.right.left.data:",root.right.left.data)

```

OUTPUT:

```

===== RES
root.right.left.data: E
Rahul_51
>

```

b. Insert nodes into a binary search tree.

CODE:

```
class TreeNode:
    def __init__(self,data):
        self.data = data
        self.left = None
        self.right = None

root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')
nodeC = TreeNode('C')
nodeD = TreeNode('D')
nodeE = TreeNode('E')
nodeF = TreeNode('F')
nodeG = TreeNode('G')
nodeH = TreeNode('H')

root.left = nodeA
root.right = nodeB

nodeA.left = nodeC
nodeA.right = nodeD

nodeB.left = nodeE
nodeB.right = nodeF

nodeF.left = nodeG
nodeF.right = nodeH
#Test
print("root.right.left.data:",root.right.left.data)
```

OUTPUT:

=====

root.right.left.data: E

Enter element to search:H

checking: R

checking: A

checking: C

checking: D

checking: B

checking: E

checking: F

checking: G

checking: H

Element Found

Rahul_51

|

c. Search for a node in a binary search tree.

CODE:

```
#search|
def search(root,key):
    if root is None:
        return False

    print("checking:",root.data)

    if root.data ==key:
        return True

    return search(root.left,key)or search(root.right,key)

key=input("Enter element to search:")

#search
if search(root,key):
    print("Element Found")
else:
    print("Element not Found")

print("Rahul_51")
```

OUTPUT:

===== REST/

root.right.left.data: E

Enter element to search:F

checking: R

checking: A

checking: C

checking: D

checking: B

checking: E

checking: F

Element Found

Rahul_51

>>

===== REST/

root.right.left.data: E

Enter element to search:S

checking: R

checking: A

checking: C

checking: D

checking: B

checking: E

checking: F

checking: G

Element not Found

Rahul_51

>>

Curiosity Questions:

1. In a BST, which side (left or right) has **smaller numbers** than the root?
2. If we insert numbers 10, 20, 5 into a BST, which number will be at the root?
3. What will the inorder traversal of a BST give — sorted numbers or random numbers?
4. Where will the **largest number** in a BST always be found?
5. If the root of a BST is 50 and we search for 70, will we go left or right? Why?
6. If a BST has only **one node**, what is its inorder traversal?
7. Can we create a BST from characters like A, B, C instead of numbers?
8. If we insert numbers 5, 10, 15, 20 in order, will the tree be balanced or like a straight line?
9. In a BST, do we check the left child first or the right child first in inorder traversal?
10. If we delete the root of a BST, will the tree disappear?